

第一阶段任务分配表

本表用于项目的起步阶段，目标不是一性把全部功能做完，而是先把主链路跑通，再逐步扩展到审核、举报、统计和后台能力。

第一阶段的判断标准很简单：前端能看到真实数据，后端能返回符合 OpenAPI 的字段，测试能把主流程和失败路径先检查一遍。

1. 阶段目标

第一阶段优先完成以下内容：

1. 微信登录与用户信息读取。
2. 商品列表、商品详情、商品发布的最小闭环。
3. 收藏、下单、订单详情、订单状态变更的基础链路。
4. 举报提交和后台最基础的管理接口。
5. 统一接口返回结构、字段命名和分页方式。

2. 开工前必须先看的文档

所有组员在动手之前，都建议先看下面 3 份文档：

- [概要设计v1.4.html](#)：看系统架构、模块划分、接口原则和状态机。
- [开发任务清单与API规范](#)：看整体任务拆分、接口总表和验收标准。
- [openapi-draft.yaml](#)：看每个接口的字段级定义、请求体、响应体和状态枚举。

如果某个人负责某一块功能，还要再补看对应代码骨架：

- 前端同学看 `pages/*`、`components/item-card`、`utils/api.js`、`utils/storage.js`。
- 后端同学看 `backend/app/main.py`、`backend/app/api/routers/*`、`backend/app/schemas/common.py`、`backend/app/core/response.py`。

3. 任务分配（按人头编号）

3.1 前端 A：主页和列表页

3.7 数据库与建模（测试同学兼任）

负责范围：

- `pages/home/index`
- `pages/list/index`
- `components/item-card`
- `utils/api.js` 的列表请求封装

第一步任务：

1. 先把首页和列表页从“空页面”改成“能拉数据并展示”的页面。
2. 首页至少展示 3 块内容：推荐商品、最新商品、分类入口。

3. 列表页至少支持分页、关键词搜索、分类筛选和刷新。
4. `item-card` 组件要能显示商品标题、价格、图片、状态标签和简单描述。
5. 先不追求复杂样式，优先保证数据能渲染出来。

需要参考的文档：

- [开发任务清单与API规范](#) 中的“前端任务清单”“API 规范”“验收标准”。
- [openapi-draft.yaml](#) 中的 `GET /api/products`、`GET /api/products/{productId}`。
- [概要设计v1.4.html](#) 中的“系统总体架构”“模块划分与职责”。

建议第一阶段交付物：

- 首页能展示商品列表数据。
- 列表页能完成一次查询和分页切换。
- `item-card` 能直接接收接口返回的数据对象。

3.2 前端 B：详情页、发布页和个人中心

负责范围：

- `pages/detail/index`
- `pages/post/index`
- `pages/user/index`
- `utils/storage.js`

第一步任务：

1. 把详情页做成能根据商品 ID 读取商品信息的页面。
2. 把个人中心页做成能读取当前用户资料、信誉分和收藏入口的页面。
3. 把发布页搭成完整表单，先实现字段校验和页面交互，再接图片上传。
4. 登录态先通过本地存储管理，不要一开始就把逻辑写散在各个页面里。
5. 把“收藏”“举报”“下单”按钮先摆出来，后面再逐步接业务逻辑。

需要参考的文档：

- [开发任务清单与API规范](#) 中的“用户与管理任务”“商品与审核任务”“交易与评价任务”。
- [openapi-draft.yaml](#) 中的 `GET /api/products/{productId}`、`GET /api/users/me`、`PUT /api/users/me`、`POST /api/products`、`POST /api/products/{productId}/images`、`POST /api/orders`。
- [概要设计v1.4.html](#) 中的“接口设计原则”“非功能与部署”。

建议第一阶段交付物：

- 商品详情页能显示完整商品信息。
- 个人中心页能显示当前用户资料。
- 发布页能完成字段校验，并预留图片上传位置。

3.3 后端 A：认证与用户

负责范围：

- `backend/app/api/routers/auth.py`

- `backend/app/api/routers/users.py`
- `backend/app/schemas/common.py` 中认证和用户相关模型
- `backend/app/core/response.py`

第一步任务：

1. 把微信登录、刷新 token、当前用户信息查询这条链路先做实。
2. 让接口返回结构严格符合统一响应格式，不要每个接口自己拼数据。
3. 把用户资料、收藏列表、收藏新增和收藏删除接口先跑通。
4. 先不做复杂权限系统，但至少把“已登录”和“未登录”区分开。
5. 尽量把字段名和 OpenAPI 对齐，避免前端联调时来回改名。

需要参考的文档：

- [开发任务清单与API规范](#) 中的“认证接口”“用户与管理接口”“统一响应结构”。
- `openapi-draft.yaml` 中的 `POST /api/auth/wx-login`、`POST /api/auth/refresh`、`GET /api/auth/me`、`GET /api/users/me`、`PUT /api/users/me`、`GET /api/users/me/favorites`。
- [概要设计v1.4.html](#) 中的“接口设计原则”“模块划分与职责”。

建议第一阶段交付物：

- 登录接口返回 token 和用户信息。
- 当前用户接口能返回资料、信誉和状态。
- 收藏接口可以先用内存数据或简单假数据跑通。

3.4 后端 B：商品与交易

负责范围：

- `backend/app/api/routers/products.py`
- `backend/app/api/routers/orders.py`
- `backend/app/schemas/common.py` 中商品和订单相关模型

第一步任务：

1. 把商品列表、商品详情、商品发布这三条主接口先做出来。
2. 把订单创建、订单详情、卖家确认、取消、完成、评价这些基础动作串起来。
3. 先保证接口能按状态流转，不要急着做特别复杂的数据库优化。
4. 商品图片上传接口可以先做成返回文件名或 URL 的最小版本。
5. 后续所有商品和订单状态都要跟 OpenAPI 里的枚举保持一致。

需要参考的文档：

- [开发任务清单与API规范](#) 中的“商品与审核任务”“交易与评价任务”“数据库设计任务”。
- `openapi-draft.yaml` 中的 `GET /api/products`、`POST /api/products`、`GET /api/products/{productId}`、`PUT /api/products/{productId}`、`POST /api/orders`、`GET /api/orders/{orderId}`、`POST /api/orders/{orderId}/seller-confirm`、`POST /api/orders/{orderId}/cancel`、`POST /api/orders/{orderId}/complete`、`POST /api/orders/{orderId}/reviews`。
- [概要设计v1.4.html](#) 中的“概念数据模型”“模块 DFD 数据流图”“后续详细设计计划”。

建议第一阶段交付物：

- 商品列表和详情接口可联调。
- 商品发布接口可返回待审核或已发布状态。
- 订单最小流程可以完成创建、确认和状态查询。

3.5 后端 C：举报、后台与统计

负责范围：

- `backend/app/api/routers/reports.py`
- `backend/app/api/routers/admin.py`
- `backend/app/schemas/common.py` 中举报、申诉、后台管理相关模型

第一步任务：

1. 先把举报提交、举报查询和申诉接口跑通。
2. 再把后台的举报队列、用户状态、商品审核和统计总览补起来。
3. 先用最基础的返回体把接口联调通，再逐步加入更完整的业务逻辑。
4. 后台统计接口先返回简单的总数或序列数据，不需要第一步就把图表做全。
5. 如果时间允许，再顺手补管理员日志结构，方便以后追踪操作。

需要参考的文档：

- [开发任务清单与API规范](#) 中的“举报与投诉任务”“统计与运维任务”。
- `openapi-draft.yaml` 中的 `POST /api/reports`、`GET /api/reports/{reportId}`、`POST /api/appeals`、`GET /api/admin/reports`、`POST /api/admin/reports/{reportId}/handle`、`GET /api/admin/stats/overview`、`GET /api/admin/stats/products`、`GET /api/admin/stats/trades`、`GET /api/admin/stats/users`、`GET /api/admin/logs`。
- [概要设计v1.4.html](#) 中的“模块划分与职责”“非功能与部署”。

建议第一阶段交付物：

- 举报提交和查询接口可联调。
- 管理后台能看到一个最基础的统计总览。
- 后续管理员处理动作有明确的字段出口。

3.6 后端 D（测试）：契约测试、联调检查与数据库建模

负责范围：

- OpenAPI 契约检查
- 主流程测试清单
- 联调问题记录
- 回归检查

第一步任务：

1. 先把 OpenAPI 按主链路拆成测试清单，明确哪些接口是登录、商品、交易、举报、后台。
2. 给每条主链路写出成功路径和失败路径的检查点。
3. 在前后端联调前先检查接口字段名、状态枚举、分页参数和响应结构。

4. 如果当前阶段独立测试岗位压力不大，可以优先协助后端 3 整理接口契约，或者协助前端 2 做页面联调验证。
5. 联调时重点记录“字段不一致”“状态不一致”“返回结构不一致”这三类问题。

需要参考的文档：

- [开发任务清单与API规范](#) 中的“API 规范”“验收标准”“团队分工与第一步任务”。
- [openapi-draft.yaml](#) 作为测试依据。
- [概要设计v1.4.html](#) 中的“接口设计原则”“关键业务流程”“非功能与部署”。

建议第一阶段交付物：

- 一份主流程联调检查表。
- 一份接口字段核对表。
- 一份问题记录表，方便后续回归。

负责范围：

- 数据库方案设计
- 表结构建模与字段命名统一
- MySQL 8.0 本地初始化
- 迁移脚本和种子数据
- 后端读写账号权限约束

第一步任务：

1. 先根据 OpenAPI 和概要设计，整理出核心数据表清单，包括用户、商品、图片、订单、收藏、评价、举报、管理员日志和统计表。
2. 使用 MySQL 8.0 作为数据库，默认端口使用 3306，先把本地数据库实例和项目的连接方式定下来。
3. 设计第一版表结构时，要把主键、外键、唯一索引、常用查询索引都考虑进去，至少保证后端后续能正常读写。
4. 为后端准备一个专门的数据库用户，权限只覆盖这个项目涉及到的表的读写操作，不要直接用 root。
5. 先输出数据库设计草案，再配合后端 1、后端 2、后端 3 逐步补充迁移脚本和初始化数据。

需要参考的文档：

- [开发任务清单与API规范](#) 中的“数据库设计任务”“后端任务清单”“验收标准”。
- [openapi-draft.yaml](#) 中所有涉及数据读写的接口，尤其是用户、商品、订单、举报和后台统计相关接口。
- [概要设计v1.4.html](#) 中的“概念数据模型”“模块划分与职责”“后续详细设计计划”。

建议第一阶段交付物：

- 一份数据库表结构草案。
- 一份 MySQL 8.0 初始化说明，写清楚 3306 端口和连接方式。
- 一份数据库用户权限说明，写清楚这个用户可以读写哪些表。
- 一份迁移和初始化脚本清单，后续后端实现时直接接上。

说明：如果后续阶段测试工作量明显增加，再考虑把数据库职责拆回给单独成员；第一阶段先让后端D兼任数据库和建模，更适合当前人手配置。

4. 推荐推进节奏

建议按下面顺序推进：

1. 后端 A 先把登录与用户信息打通。
2. 前端 B 先把登录态和个人中心接起来。
3. 后端 B 先把商品列表和详情打通。
4. 前端 A 先把首页和列表页接上。
5. 后端 B 再把下单和订单状态补出来。
6. 后端 C 和后端 D 一起把举报、后台和数据库最小链路整理出来。

这样做的好处是，最早一轮就能看到“登录-浏览-发布-下单-举报”这一整条业务主线开始有结果，而不是所有人都只在写空壳。